

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr.T.P.Anithaashri

QUESTIONS: 10

Total Marks: 50

Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Code of Conduct:

I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



K. Vijay Bhaskar Reddy

Experiment 1

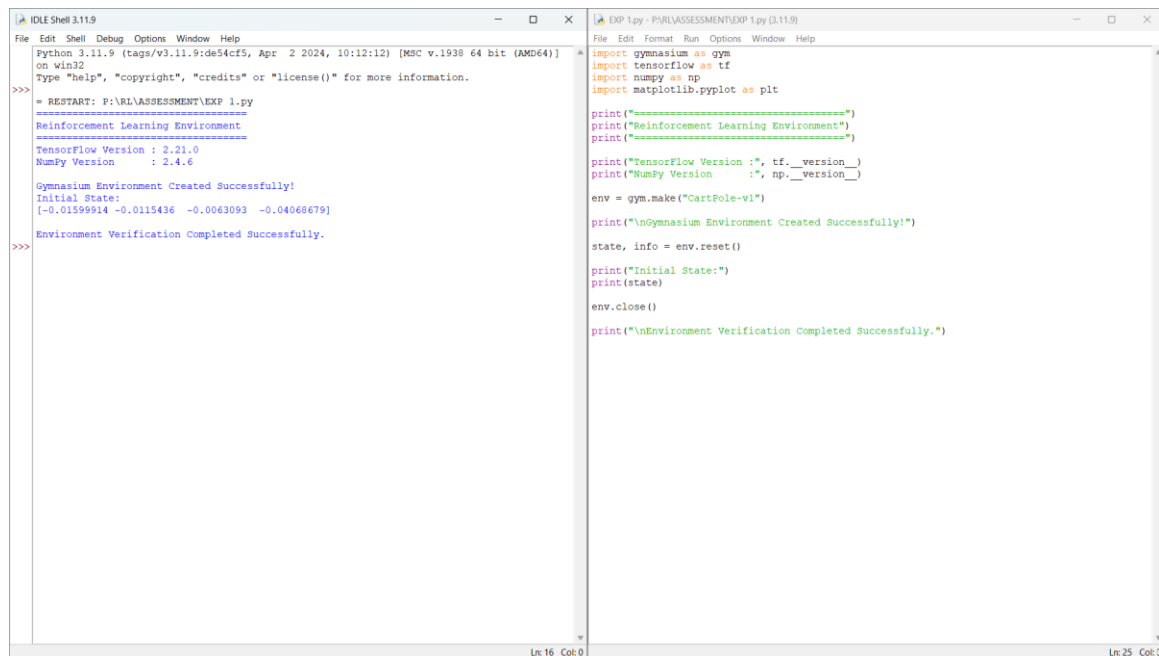
Title: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Python, TensorFlow, Keras, Gymnasium, NumPy and Matplotlib and verify the RL environment.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



```
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
> RESTART: F:\RL\ASSESSMENT\EXP 1.py
=====
Reinforcement Learning Environment
=====
TensorFlow Version : 2.21.0
NumPy Version : 2.4.6
Gymnasium Environment Created Successfully!
Initial State:
[-0.01599914 -0.0115436 -0.0063093 -0.04068679]
Environment Verification Completed Successfully.
>>>

File Edit Format Run Options Window Help
EXP 1.py - PYRLASSESSMENT\EXP 1.py (3.11.9)
import gymnasium as gym
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("=====")
print("Reinforcement Learning Environment")
print("=====")

print("TensorFlow Version :", tf.__version__)
print("NumPy Version      :", np.__version__)

env = gym.make("CartPole-v1")

print("\nGymnasium Environment Created Successfully!")

state, info = env.reset()

print("Initial State:")
print(state)

env.close()

print("\nEnvironment Verification Completed Successfully.")
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 2

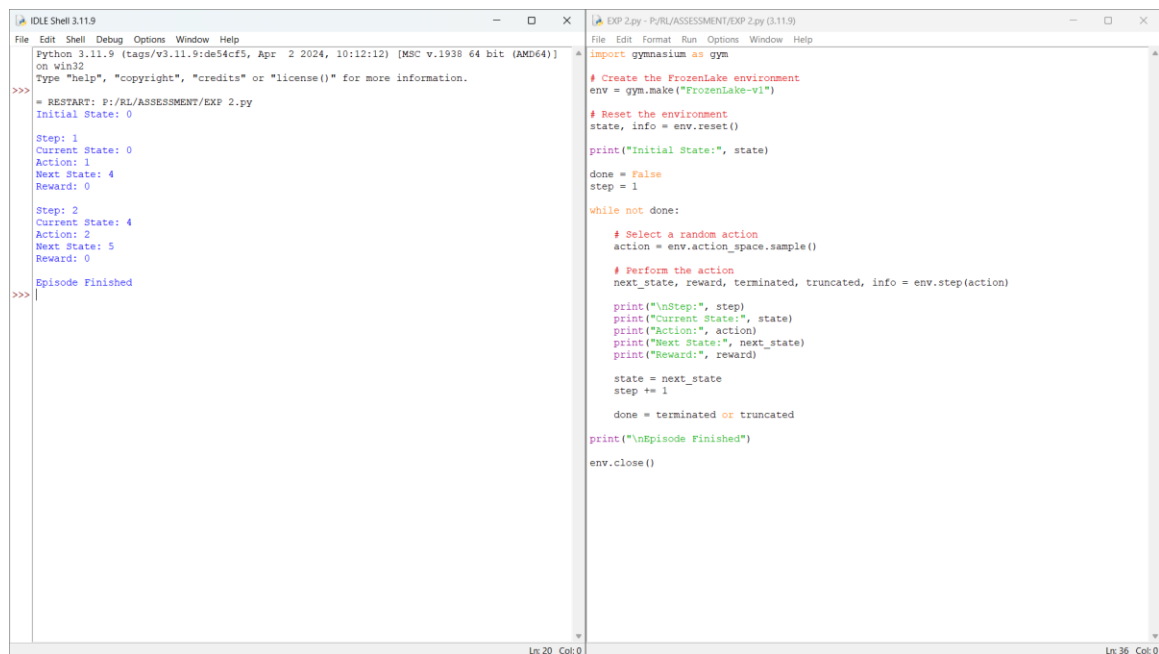
Title: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create and interact with the FrozenLake environment and understand states, actions, rewards and episodes.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



```
Python Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: F:/RL/ASSESSMENT/EXP 2.py
Initial State: 0

Step: 1
Current State: 0
Action: 1
Next State: 4
Reward: 0

Step: 2
Current State: 4
Action: 2
Next State: 5
Reward: 0

Episode Finished
>>>
```

```
EXP 2.py - P:/RL/ASSESSMENT/EXP 2.py (3.11.9)
File Edit Format Run Options Window Help
import gymnasium as gym

# Create the FrozenLake environment
env = gym.make("FrozenLake-v1")

# Reset the environment
state, info = env.reset()

print("Initial State:", state)

done = False
step = 1

while not done:

    # Select a random action
    action = env.action_space.sample()

    # Perform the action
    next_state, reward, terminated, truncated, info = env.step(action)

    print("\nStep:", step)
    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    state = next_state
    step += 1

    done = terminated or truncated

print("\nEpisode Finished")

env.close()
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 3

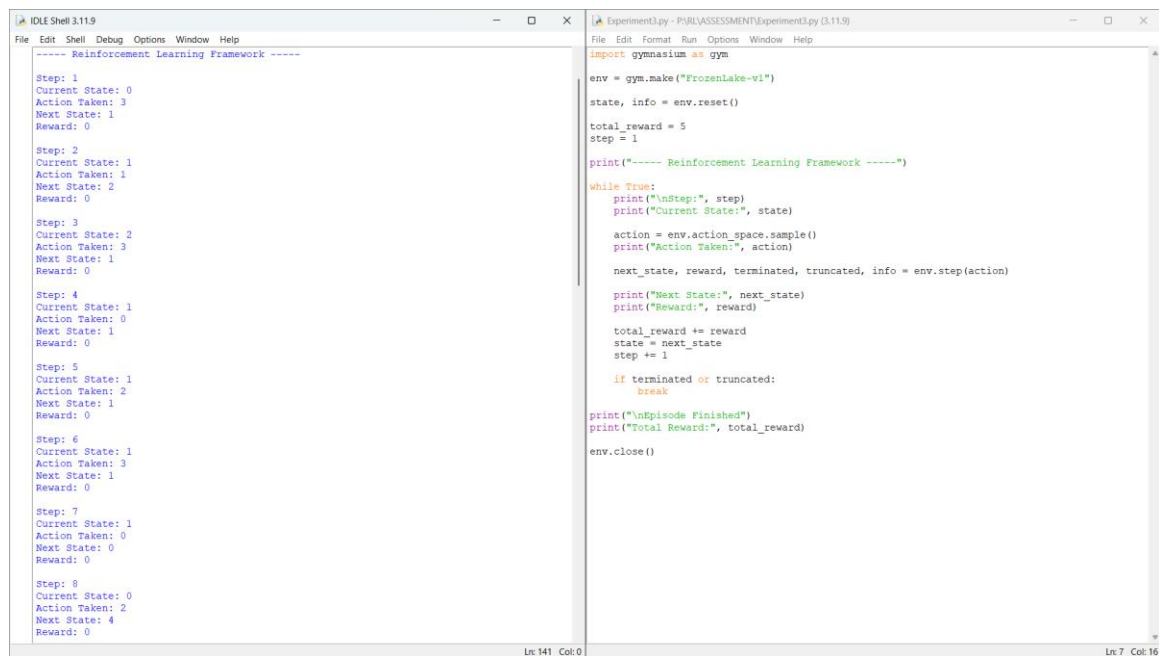
Title: Implementation of Reinforcement Learning Framework

Aim: To implement the basic RL framework using an agent interacting with the environment.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



The image shows two side-by-side windows from an IDE. The left window, titled 'IDLE Shell 3.11.9', displays the output of a Python script. It shows a sequence of 8 steps, each with 'Current State', 'Action Taken', 'Next State', and 'Reward' values. The right window, titled 'Experiment3.py - P:\RL\ASSESSMENT\Experiment3.py (3.11.9)', shows the corresponding Python code. The code imports 'gymnasium as gym', creates a 'FrozenLake-v1' environment, resets it, and enters a loop where it takes actions, receives states and rewards, and updates the total reward. The loop breaks when terminated or truncated, and finally prints the total reward and closes the environment.

```
----- Reinforcement Learning Framework -----  
Step: 1  
Current State: 0  
Action Taken: 3  
Next State: 1  
Reward: 0  
Step: 2  
Current State: 1  
Action Taken: 1  
Next State: 2  
Reward: 0  
Step: 3  
Current State: 2  
Action Taken: 3  
Next State: 1  
Reward: 0  
Step: 4  
Current State: 1  
Action Taken: 0  
Next State: 1  
Reward: 0  
Step: 5  
Current State: 1  
Action Taken: 2  
Next State: 1  
Reward: 0  
Step: 6  
Current State: 1  
Action Taken: 3  
Next State: 1  
Reward: 0  
Step: 7  
Current State: 1  
Action Taken: 0  
Next State: 0  
Reward: 0  
Step: 8  
Current State: 0  
Action Taken: 2  
Next State: 4  
Reward: 0
```

```
import gymnasium as gym  
  
env = gym.make("FrozenLake-v1")  
state, info = env.reset()  
total_reward = 0  
step = 1  
  
print("----- Reinforcement Learning Framework -----")  
while True:  
    print("\nStep:", step)  
    print("Current State:", state)  
    action = env.action_space.sample()  
    print("Action Taken:", action)  
    next_state, reward, terminated, truncated, info = env.step(action)  
    print("Next State:", next_state)  
    print("Reward:", reward)  
    total_reward += reward  
    state = next_state  
    step += 1  
    if terminated or truncated:  
        break  
print("\nEpisode Finished")  
print("Total Reward:", total_reward)  
env.close()
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 4

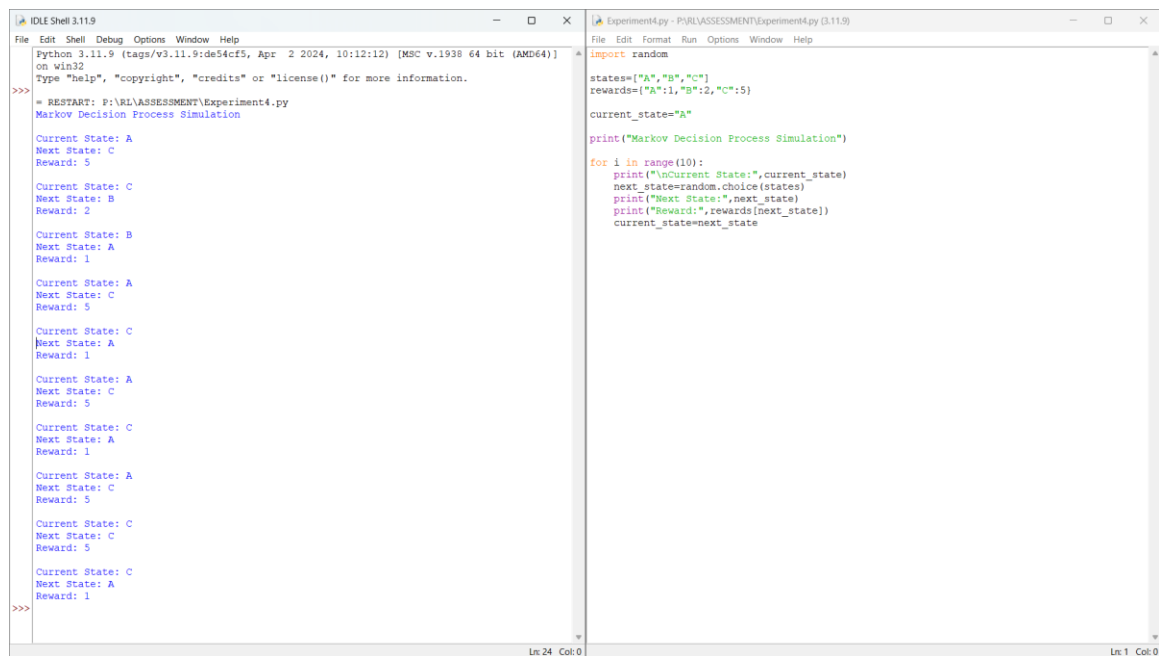
Title: Simulation of Markov Decision Process (MDP)

Aim: To simulate state transitions, rewards and policies using a simple MDP.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\VRL\ASSESSMENT\Experiment4.py
Markov Decision Process Simulation

Current State: A
Next State: C
Reward: 5

Current State: C
Next State: B
Reward: 2

Current State: B
Next State: A
Reward: 1

Current State: A
Next State: C
Reward: 5

Current State: C
Next State: A
Reward: 1

Current State: A
Next State: C
Reward: 5

Current State: C
Next State: A
Reward: 1

Current State: A
Next State: C
Reward: 5

Current State: C
Next State: A
Reward: 1

Current State: A
Next State: C
Reward: 5

Current State: C
Next State: C
Reward: 5

Current State: C
Next State: C
Reward: 5

Current State: C
Next State: A
Reward: 1

>>>
```

```
Experiment4.py - P:\VRL\ASSESSMENT\Experiment4.py (3.11.9)
File Edit Format Run Options Window Help
import random

states=["A","B","C"]
rewards={"A":1,"B":2,"C":5}

current_state="A"

print("Markov Decision Process Simulation")

for i in range(10):
    print("\nCurrent State:",current_state)
    next_state=random.choice(states)
    print("Next State:",next_state)
    print("Reward:",rewards[next_state])
    current_state=next_state
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 5

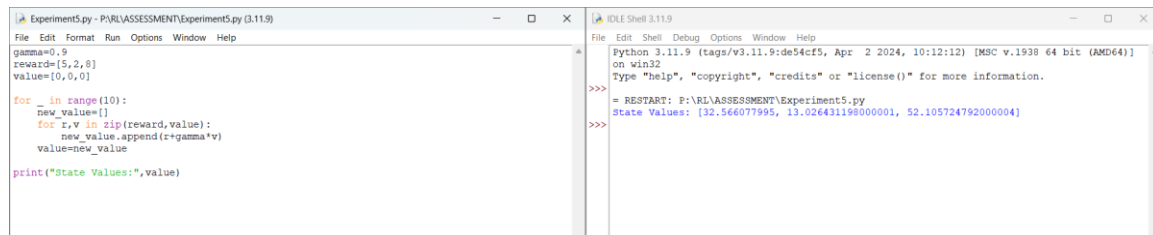
Title: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality equations.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



The screenshot shows an IDE with two windows. The left window, titled 'Experiment5.py - P:\RL\ASSESSMENT\Experiment5.py (3.11.9)', contains the following Python code:

```
gamma=0.9
reward=[5,2,8]
value=[0,0,0]

for _ in range(10):
    new_value=[]
    for i,v in zip(reward,value):
        new_value.append((+gamma*v)
    value=new_value

print("State Values:",value)
```

The right window, titled 'IDLE Shell 3.11.9', shows the output of the script:

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\Experiment5.py
State Values: [32.566077995, 13.026431198000001, 52.105724792000004]
>>>
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 6

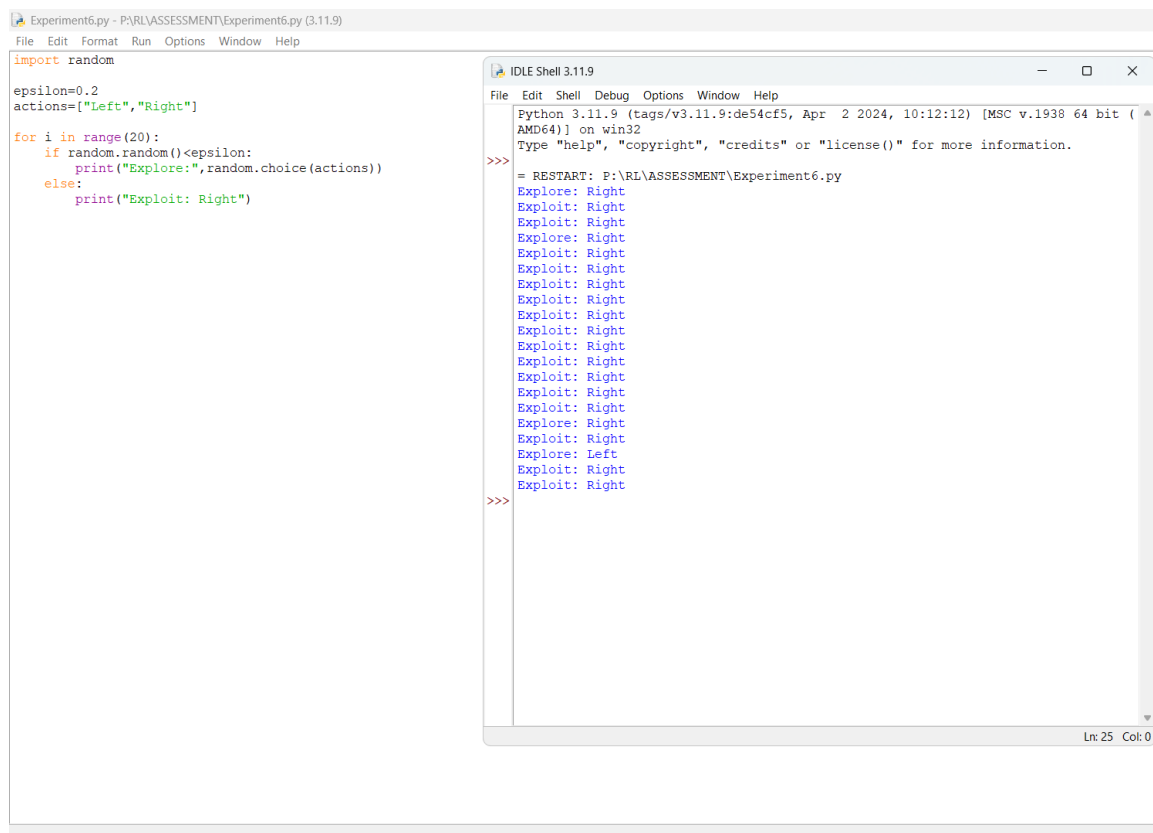
Title: Exploration vs Exploitation

Aim: To compare Greedy, ϵ -Greedy and Random strategies.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



```
Experiment6.py - P:\RL\ASSESSMENT\Experiment6.py (3.11.9)
File Edit Format Run Options Window Help

import random

epsilon=0.2
actions=["Left","Right"]

for i in range(20):
    if random.random()<epsilon:
        print("Explore:",random.choice(actions))
    else:
        print("Exploit: Right")

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\Experiment6.py
Explore: Right
Exploit: Right
Exploit: Right
Exploit: Right
Explore: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Exploit: Right
Explore: Right
Exploit: Right
Explore: Left
Exploit: Right
Exploit: Right
>>>
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 7

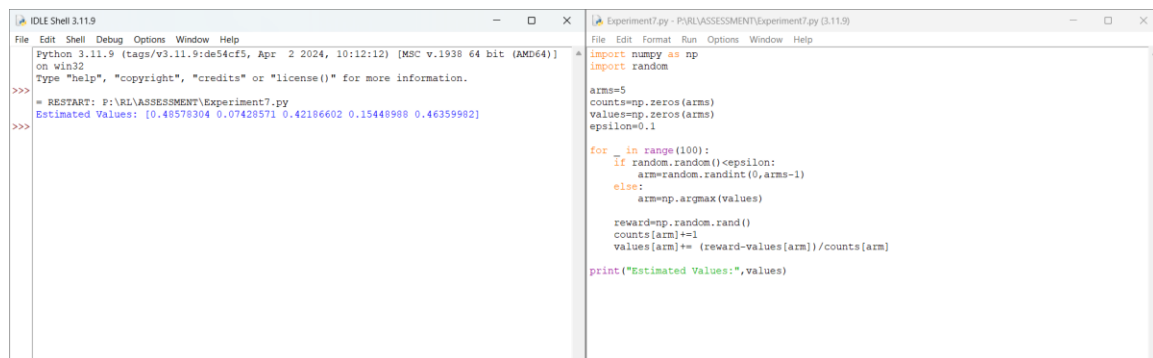
Title: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



The screenshot displays two windows from an IDE. The left window, titled 'IDLE Shell 3.11.9', shows the execution output of a Python script. It starts with a prompt 'Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32'. After a restart, it displays the output: 'Estimated Values: [0.48578304 0.07428571 0.42186602 0.15448988 0.46359982]'. The right window, titled 'Experiment7.py - P:\RL\ASSESSMENT\Experiment7.py (3.11.9)', shows the source code of the script. The code imports 'numpy as np' and 'random'. It initializes 'arms=5', 'counts=np.zeros(arms)', 'values=np.zeros(arms)', and 'epsilon=0.1'. A loop runs 100 times, where in each iteration, a random number is generated. If it is less than 'epsilon', a random arm is selected. Otherwise, the arm with the highest current value is selected. The reward is then added to the count and value of the selected arm. Finally, the estimated values are printed.

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\RL\ASSESSMENT\Experiment7.py
Estimated Values: [0.48578304 0.07428571 0.42186602 0.15448988 0.46359982]
>>>

File Edit Shell Debug Options Window Help
Experiment7.py - P:\RL\ASSESSMENT\Experiment7.py (3.11.9)
File Edit Format Run Options Window Help

import numpy as np
import random

arms=5
counts=np.zeros(arms)
values=np.zeros(arms)
epsilon=0.1

for _ in range(100):
    If random.random()<epsilon:
        arm=random.randint(0,arms-1)
    else:
        arm=np.argmax(values)

    reward=np.random.rand()
    counts[arm]+=1
    values[arm]+=(reward-values[arm])/counts[arm]

print("Estimated Values:",values)
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 8

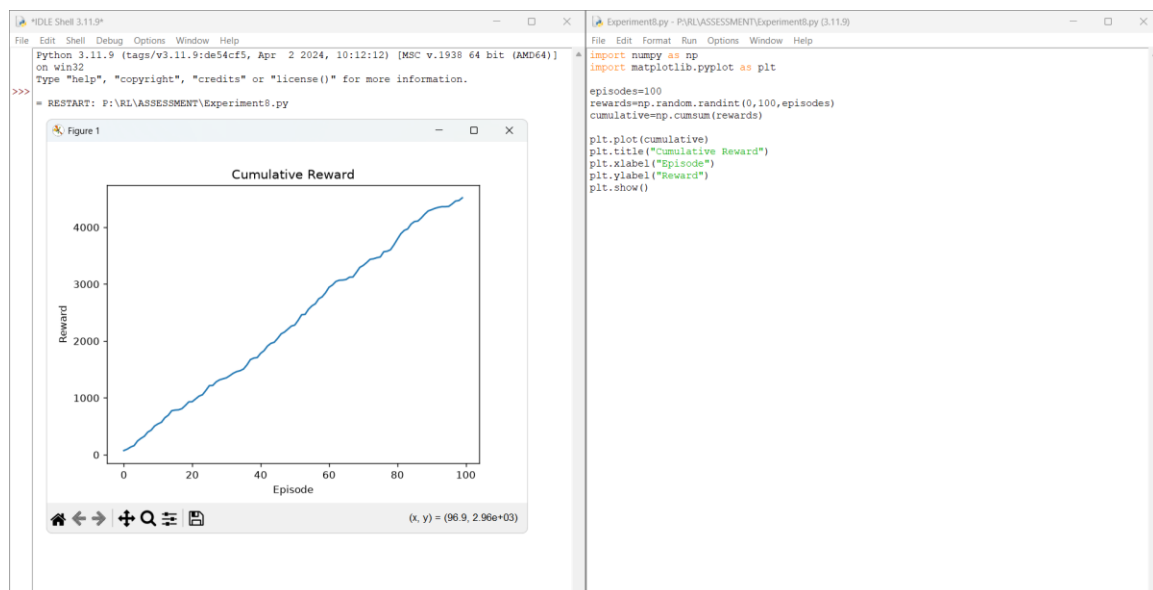
Title: Reward Visualization

Aim: To visualize cumulative rewards using Matplotlib.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



Result: The experiment was executed successfully and the expected output was obtained.

Experiment 9

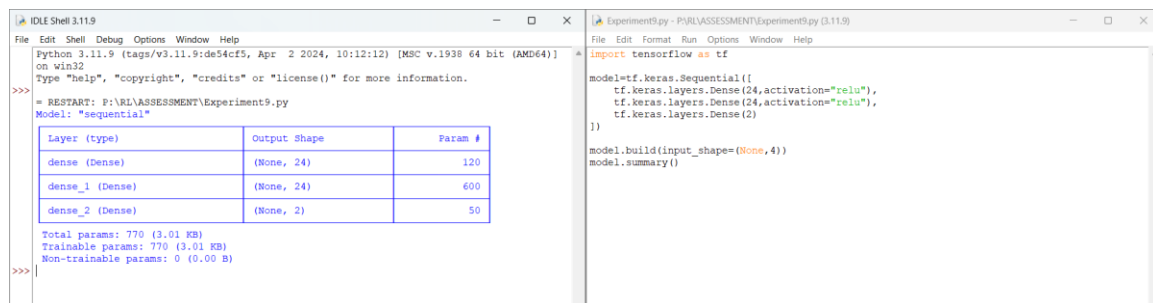
Title: TensorFlow and Keras Neural Network

Aim: To build a simple feed-forward neural network for RL.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



The screenshot shows an IDE with two windows. The left window, titled 'IDLE Shell 3.11.9', displays the output of a Python script. It shows the model name 'sequential' and a table of layer details. The right window, titled 'Experiment9.py - P:\RL\ASSESSMENT\Experiment9.py (3.11.9)', shows the Python code used to create the model.

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: P:\RL\ASSESSMENT\Experiment9.py
Model: "sequential"

```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 24)	120
dense_1 (Dense)	(None, 24)	600
dense_2 (Dense)	(None, 2)	50

```
Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)
>>>
```

```
Experiment9.py - P:\RL\ASSESSMENT\Experiment9.py (3.11.9)
import tensorflow as tf

model=tf.keras.Sequential([
    tf.keras.layers.Dense(24,activation="relu"),
    tf.keras.layers.Dense(24,activation="relu"),
    tf.keras.layers.Dense(2)
])

model.build(input_shape=(None,4))
model.summary()
```

Result: The experiment was executed successfully and the expected output was obtained.

Experiment 10

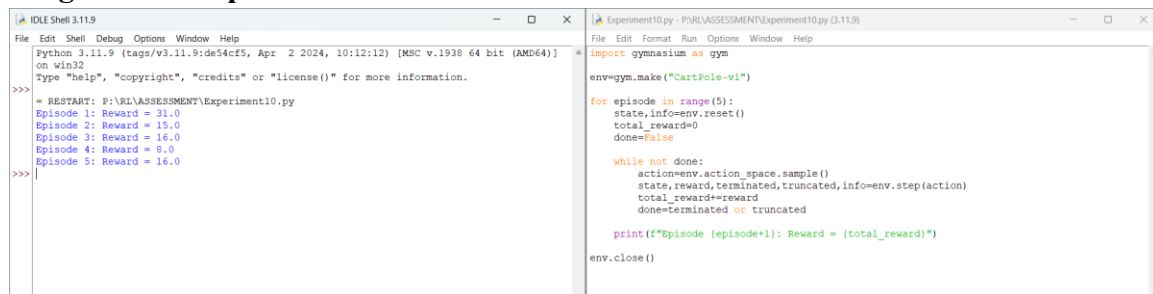
Title: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic RL agent for the CartPole environment.

Algorithm:

- Import the required libraries.
- Initialize the environment/model.
- Execute the required RL procedure.
- Display the output.
- Verify the result.

Program & Output:



The screenshot displays a Python IDE with two windows. The left window, titled 'IDLE Shell 3.11.9', shows the execution output of the program. It starts with a restart message and then displays the results for five episodes: Episode 1: Reward = 31.0, Episode 2: Reward = 15.0, Episode 3: Reward = 16.0, Episode 4: Reward = 8.0, and Episode 5: Reward = 16.0. The right window, titled 'Experiment10.py - Python 3.11.9', contains the source code. The code imports the 'gymnasium' library as 'gym', creates a 'CartPole-v1' environment, and runs a loop for 5 episodes. In each episode, it resets the environment, samples an action, and steps the environment until it is terminated or truncated, accumulating the total reward. A print statement at the end of each episode shows the total reward, which matches the output in the shell window.

```
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: F:\RL\ASSESSMENT\Experiment10.py
Episode 1: Reward = 31.0
Episode 2: Reward = 15.0
Episode 3: Reward = 16.0
Episode 4: Reward = 8.0
Episode 5: Reward = 16.0
>>>

File Edit Format Run Options Window Help
import gymnasium as gym

env=gym.make("CartPole-v1")

for episode in range(5):
    state,info=env.reset()
    total_reward=0
    done=False

    while not done:
        action=env.action_space.sample()
        state,reward,terminated,truncated,info=env.step(action)
        total_reward+=reward
        done=terminated or truncated

    print(f"Episode {episode+1}: Reward = {total_reward}")

env.close()
```

Result: The experiment was executed successfully and the expected output was obtained.